

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 05 -

EXPLORANDO FALHAS: INJECTIONS E XSS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	13
O QUE VOCÊ VIU NESTA AULA?	14
REFERÊNCIAS.....	15

O QUE VEM POR AÍ?

Você já solidificou a segurança do seu código-fonte através de análises estáticas (SAST). No entanto, um código perfeito não garante uma aplicação segura, pois falhas de segurança frequentemente residem na arquitetura de *runtime*, nas configurações de nuvem e na interação dinâmica entre componentes. É aqui que entra o Dynamic Application Security Testing (DAST).

Nesta aula, vamos mudar a perspectiva de "olhar para dentro do código" para "atacar a aplicação como um invasor". Você será guiado(a) pela automação da fase de *pentest* utilizando o OWASP ZAP, a ferramenta líder de mercado, integrando-a ao seu *pipeline* de CI/CD para validar que o *deploy* em ambientes AWS não introduza vulnerabilidades exploráveis. Prepare-se para implementar um *gate* de segurança robusto que opera em tempo real, garantindo que a velocidade da entrega não comprometa a integridade do sistema.

HANDS ON

A experiência de aprendizagem nesta aula está estruturada para que você mova a segurança para a direita, validando o comportamento da aplicação em execução. Nas videoaulas, você começará focando na preparação do ambiente de *staging* na AWS e na configuração inicial do OWASP ZAP, estabelecendo o alvo de *runtime* que será submetido à varredura. Em seguida, o foco será a automação da segurança, onde você implementará o ZAP Baseline Scan via GitHub Actions contra a aplicação rodando na AWS. Por fim, consolidará o aprendizado no ciclo de correção (*Fix & Verify*): você analisará os alertas críticos gerados, aplicará a correção no código e reexecutará o *pipeline* para validação da segurança. Este ciclo prático garantirá a consolidação do aprendizado, reforçando o *feedback loop* essencial do DevSecOps.

SAIBA MAIS

DAST no DevSecOps: A Convergência Estratégica de Shift Left e Shift Right

O Dynamic Application Security Testing (DAST) é uma metodologia de teste de segurança que avalia uma aplicação em seu estado de execução, simulando o comportamento de um atacante sem acesso ao código-fonte subjacente. Essa abordagem, conhecida como teste *black-box*, é um componente insubstituível na estratégia DevSecOps, pois preenche as lacunas deixadas por análises que operam exclusivamente no código.

O **DAST** é o complemento essencial ao Static Application Security Testing (SAST). O **SAST** adota o princípio de *Shift Left*, focado em identificar falhas de código estruturais e problemas de sintaxe precocemente, durante o desenvolvimento ou *build*. Em contraste, o DAST opera como um *Shift Right* (movendo a segurança para a direita no ciclo de vida do desenvolvimento), pois só pode ser executado quando a aplicação está implantada em um ambiente ativo, como *staging* ou pré-produção. Essa diferença no *timing* e no contexto de execução é fundamental.

A principal contribuição do DAST é a validação de segurança em *runtime* e a descoberta de vulnerabilidades que surgem da interação entre o código, a infraestrutura e a configuração do ambiente. Enquanto o SAST se destaca na identificação de falhas de código (como injeção de código ou uso de funções inseguras), o DAST é insubstituível para detectar:

- **Configurações Incorretas:** o principal vetor de ataque em ambientes *cloud-native*. O DAST valida se *security headers* estão faltando ou se há exposição acidental de *endpoints* ou informações sensíveis.
- **Broken Authentication e Sessão:** falhas no fluxo de autenticação, tratamento fraco de sessões ou autorização que só se manifestam quando o sistema está operando.
- **Vulnerabilidades de API:** o DAST testa as interfaces expostas (*endpoints*, *request/response behavior*), simulando o que um ator malicioso pode realmente alcançar.

Uma vantagem crítica do DAST é a sua capacidade de reduzir a incidência de falsos positivos em comparação com o SAST. Isso se deve ao fato de que, quando o DAST dispara um alerta, ele está validando o risco em um caminho de ataque real e explícito no ambiente implantado. Se a ferramenta consegue explorar uma falha (como injetar um *payload* malicioso e obter uma resposta específica), o risco não é teórico, mas sim uma ameaça real e explorável. Esta validação de contexto é crucial para que os arquitetos de segurança e engenheiros de DevOps priorizem o trabalho de correção de forma eficiente.

Critério	SAST (Static Application Security Testing)	DAST (Dynamic Application Security Testing)
Mecanismo	Análise de código-fonte/binários (White-Box).	Simulação de ataques em aplicação rodando (Black-Box).
Fase SDLC	Shift Left (Desenvolvimento, Build).	Shift Right (Staging, Runtime, Pré-Produção).
Visibilidade	Estrutura Interna, Fluxos de Dados e Lógica.	Superfície de Ataque Exposta (Endpoints, APIs).
Vulnerabilidades Típicas	Flaws de Código (Injeção de Código, Funções Inseguras, Lógicas).	Falhas de Configuração, Broken Authentication, Falhas de API, XSS, SQLi.
Risco de Falsos Positivos	Mais Alto (devido à falta de contexto de execução).	Mais Baixo (valida ameaças reais em runtime).

Tabela 1 – Comparativo SAST vs. DAST
Fonte: Elaborado pelo autor (2025)

DevSecOps Architecture Diagram

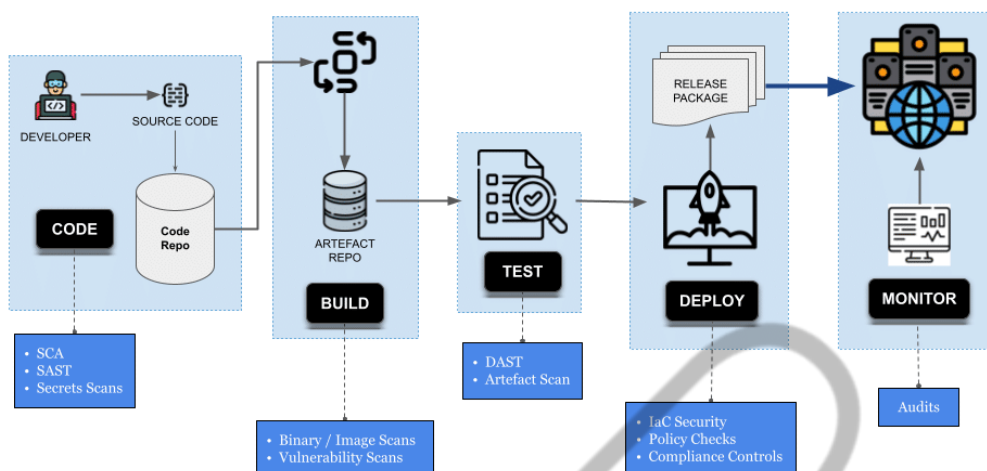


Figura 1 – DAST DevSecOps

Fonte: [Opsmx](#) (2025)

O OWASP ZAP: Passive Scan e Active Scan

O OWASP Zed Attack Proxy (ZAP) é uma das ferramentas de segurança de código aberto mais utilizadas, além de ser central para a automação do DAST. O ZAP pode ser executado em modo *headless* via API ou Docker, tornando-o perfeito para integração em *pipelines* de CI/CD. Para realizar uma varredura abrangente, o ZAP emprega duas técnicas distintas: o Passive Scanning e o Active Scanning.

Passive Scanning (Passive Scan Rules)

O *Passive Scanning* é um mecanismo de escaneamento que opera de forma não-intrusiva e automática em segundo plano. Ele funciona monitorando o tráfego HTTP (requisições e respostas) que já está passando pela aplicação, sem enviar novos *requests* ou modificar os dados.

O Passive Scan tem um impacto operacional muito baixo, sendo ideal para ser executado continuamente no *pipeline* sem atrasar o processo de *build* ou *deploy*. O foco deste mecanismo não está na exploração profunda, mas sim na detecção de vulnerabilidades que são facilmente observáveis no tráfego ou na configuração da aplicação, como a falta de *security headers* (por exemplo, Content Security Policy - CSP ou HTTP Strict Transport Security - HSTS) ou a divulgação acidental de informações sensíveis em URLs ou mensagens de erro. A ação `zapproxy/action-`

baseline no GitHub Actions, por exemplo, utiliza o Passive Scan para uma verificação rápida e segura, servindo como um *gate* de segurança contínuo em todo *commit*.

Active Scanning (Pentest Automatizado)

O *Active Scanning*, por outro lado, é altamente intrusivo e agressivo. Este mecanismo envolve a injeção de *payloads* maliciosos e a modificação ativa das requisições registradas para simular ataques e observar a reação do servidor. O Active Scan busca falhas profundas que exigem interação direta com a aplicação, como *Cross-Site Scripting* (XSS), *SQL Injection* (SQLi), *Server-Side Request Forgery* (SSRF) e outros.

Devido à sua natureza disruptiva, que pode sobrecarregar sistemas ou até mesmo afetar dados, o Active Scan exige restrições operacionais rígidas. Ele deve ser executado exclusivamente em ambientes de teste controlados e isolados (como um ambiente de *staging* na AWS), nunca em produção. Uma estratégia DAST madura, que os engenheiros de DevOps e arquitetos devem adotar, consiste em usar o Passive Scan de forma contínua e rápida em todos os *commits*, e reservar o Active Scan para varreduras intermitentes, mas profundas, em *releases* importantes ou *nightly builds* no ambiente isolado. Este balanceamento garante o *feedback* rápido sem sacrificar a profundidade da análise de segurança.

Estratégias de Varredura Autenticada: Integrando ZAP com Testes Funcionais

Um dos maiores desafios técnicos do DAST é a varredura de aplicações que exigem autenticação de usuário. Se o scanner DAST opera como um atacante não autenticado por padrão, ele só conseguirá mapear a superfície pública do aplicativo, deixando a maior parte da lógica de negócio e as áreas restritas sem teste.

A solução avançada para contornar isso é integrar o OWASP ZAP com *frameworks* de teste funcional ponta a ponta (E2E), como Selenium ou Cypress. Essa abordagem utiliza os scripts de teste funcional, que já foram desenvolvidos para simular o comportamento de um usuário autenticado, para guiar o ZAP através de toda a aplicação.

O fluxo técnico dessa integração opera da seguinte forma:

1. **Início do ZAP Daemon:** o ZAP é iniciado em modo *headless* (daemon) e configurado para atuar como um *proxy* interceptador, aguardando o tráfego da aplicação.
2. **Roteamento de Tráfego:** os scripts de teste funcional (por exemplo, Cypress) são configurados para rotear todo o seu tráfego através do *proxy* do ZAP.
3. **Navegação e Passive Scan:** o script funcional executa o login (autenticando-se com sucesso) e navega por todas as áreas restritas da aplicação. Enquanto isso, o ZAP executa seu Passive Scan em segundo plano, registrando e mapeando a "árvore de sítio" autenticada, aprendendo todos os *endpoints* restritos com as sessões de usuário válidas.
4. **Orquestração e Active Scan:** após a conclusão da varredura passiva (verificada através de *endpoints* da API ZAP que monitoram o *status* da varredura), o *pipeline* utiliza a API do ZAP para disparar um Active Scan direcionado. O *Active Scan* é então executado contra o escopo de URLs que o ZAP "aprendeu" durante a navegação autenticada.

Ao utilizar a cobertura dos testes E2E como o motor para a cobertura de segurança DAST, a equipe garante que a profundidade da varredura seja diretamente proporcional à qualidade dos scripts de teste funcionais. Isso reforça um princípio central do DevSecOps: segurança e qualidade são metas conjuntas e codependentes, onde o investimento em um beneficia diretamente o outro.

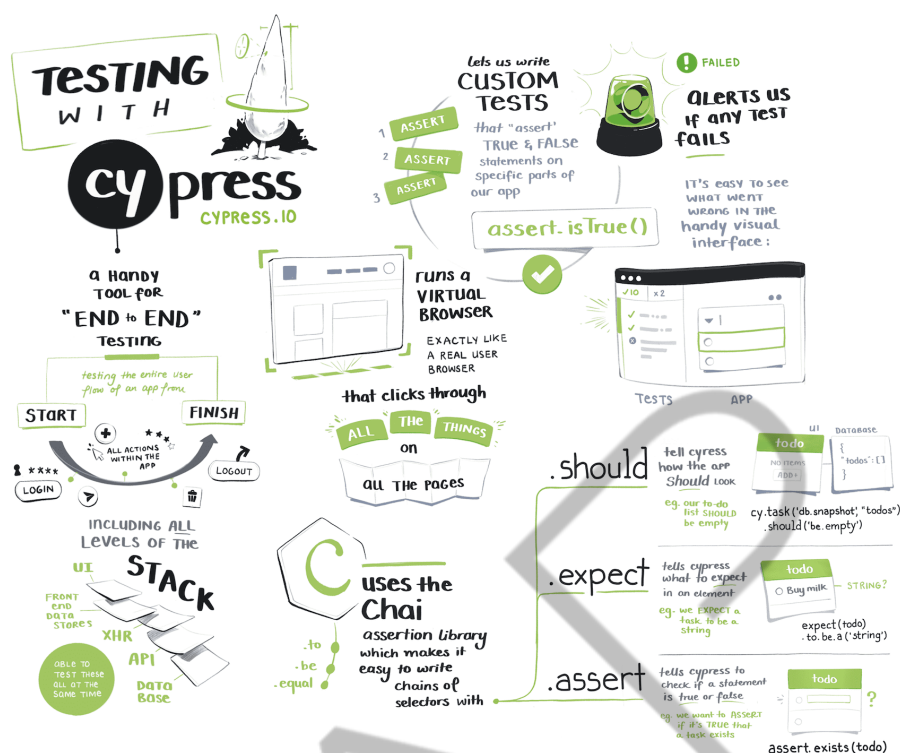


Figura 2 – Teste E2E Cypress

Fonte: [Cypress-io](https://www.cypress.io) (2025)

DevSecOps Prático: Integração DAST em Pipelines Cloud

Para um engenheiro de DevOps e arquiteto *cloud-native*, a integração do DAST deve ser automatizada e realizada contra um ambiente que mimetize a produção. Em um contexto AWS, o alvo ideal é um ambiente de *staging* vivo (como Elastic Beanstalk, ECS Fargate, ou API Gateway).

A automação no *pipeline* de CI/CD, como via GitHub Actions, simplifica a execução do ZAP. O fluxo típico para o *ZAP Baseline Scan* (Passive Scan) na AWS envolve:

1. **Implantação no Staging:** o código é construído e implantado automaticamente no ambiente de *staging* da AWS. É crucial que este ambiente esteja funcional e acessível para a varredura dinâmica.
2. **Execução da Action ZAP:** uma ação, como *zapproxy/action-baseline*, é invocada no *pipeline*. O URL do ambiente de *staging* na AWS é fornecido como *target*.

3. **Gate de Segurança:** a ação ZAP é configurada para falhar o *pipeline* (FAIL) se forem encontradas vulnerabilidades críticas que violem um arquivo de regras pré-definido. Se o *scan* for limpo, o *pipeline* pode seguir para a próxima etapa (por exemplo, aprovação manual ou *deploy* em produção).

A execução do DAST na nuvem oferece um benefício que vai além da simples varredura de código: ele valida a segurança operacional da Infraestrutura como Código (IaC). Uma falha DAST em *runtime* pode não ser causada por um bug de código, mas sim por uma configuração incorreta de infraestrutura, como um *Security Group* da AWS excessivamente permissivo, um *IAM Role* com privilégios de mais, ou um *endpoint* de API Gateway exposto. O DAST complementa, portanto, a varredura estática de IaC, validando o comportamento dinâmico e operacional da pilha de nuvem implantada. Desta forma, o engenheiro de DevOps obtém uma camada final de garantia de que o ambiente está seguro *em uso*.

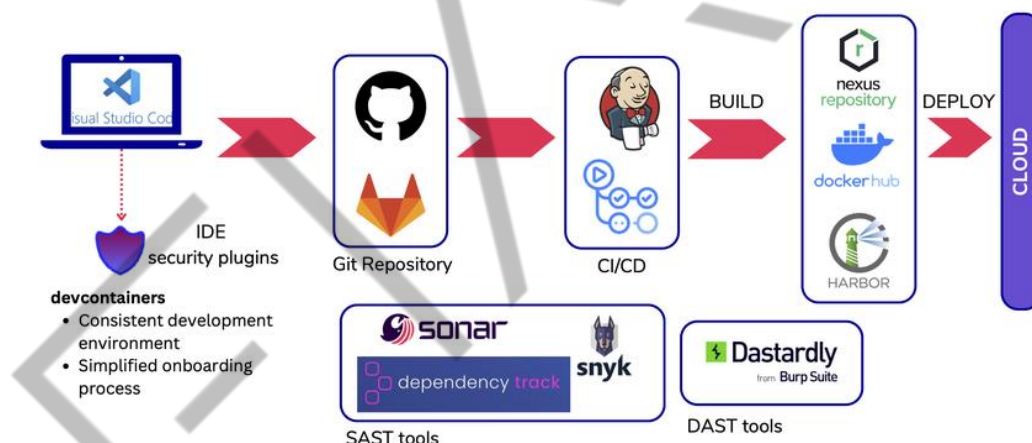


Figura 3 – DAST na Pipeline CI/CD

Fonte: [Cypress-io](#) (2025)

Gestão de Vulnerabilidades: O Ciclo de Correção e Validação (Fix & Verify)

Em DevSecOps, identificar uma vulnerabilidade é apenas metade do trabalho; a outra metade é garantir que a correção seja aplicada e verificada. O ciclo de correção e validação (*Fix & Verify*) é o mecanismo pelo qual a segurança é continuamente melhorada.

O principal desafio após a varredura DAST é transformar o alerta técnico (exemplo: "SQL Injection detectado no parâmetro X do *endpoint* Y") em uma ação de

desenvolvimento específica. Ferramentas DAST, como o ZAP, fornecem a Prova de Conceito (PoC), que é vital para que a pessoa desenvolvedora possa replicar a falha em seu ambiente local e aplicar a correção de código precisa.

A automação desse ciclo é a chave para a velocidade:

1. O desenvolvedor aplica a correção no código-fonte para mitigar o vetor de ataque.
2. O *commit* aciona o *pipeline* de CI/CD.
3. A nova versão da aplicação é implantada no ambiente de *staging*.
4. O DAST (ou um teste de regressão de segurança focado no alerta anterior) é reexecutado automaticamente.
5. Se o alerta de segurança desaparecer, a correção é verificada (*Verified*), e o *pipeline* pode seguir em frente.

Para líderes técnicos e arquitetos de projetos, a gestão do DAST deve ser orientada por dados. A consolidação dos resultados de segurança de todas as ferramentas de teste (SAST, DAST, SCA) em um repositório centralizado (como um banco de dados) permite o monitoramento contínuo de KPIs críticos. Isso inclui métricas como o Tempo Médio para Correção (MTTR) de vulnerabilidades críticas de *runtime* e a densidade de vulnerabilidades por *release*. Ao medir esses dados, a organização transforma o ZAP de uma ferramenta pontual em um motor para a melhoria contínua da cultura de segurança da engenharia. **É fundamental que você implemente essa cultura de métricas em seus projetos (Call to Action)**, pois a segurança baseada em evidências é a única que se sustenta em escala.

MERCADO, CASES E TENDÊNCIAS

A adoção de testes dinâmicos de segurança (DAST) deixou de ser uma prática isolada para se tornar um requisito mandatório no mercado moderno, especialmente em ambientes *cloud-native*. A transição para arquiteturas descentralizadas, como microsserviços e funções *serverless*, aumentou exponencialmente a superfície de ataque e a complexidade das configurações de *runtime*. Nesses ambientes dinâmicos, o DAST é o único meio de garantir que a composição complexa de serviços e as configurações de nuvem não introduzam vulnerabilidades exploráveis.

Contextualização e Redução da Fadiga de Alertas

O excesso de alertas (*alert fatigue*) é um problema conhecido em ambientes de segurança, onde a inundação de notificações de baixo impacto desvia a atenção dos profissionais de DevOps e segurança. A próxima onda de DAST está focada em resolver isso através da contextualização de achados.

Plataformas de segurança mais sofisticadas estão utilizando a inteligência artificial (AI/ML) para correlacionar os resultados do DAST (falhas de *runtime*) com os achados do SAST (falhas de código) e do SCA (*Software Composition Analysis*). O objetivo é apresentar ao desenvolvedor apenas as vulnerabilidades de alto risco que foram comprovadamente exploráveis em *runtime* e cuja origem no código-fonte é conhecida. Essa inteligência reduz significativamente o ruído, transformando uma lista extensa de potenciais problemas em uma lista curta e priorizada de ameaças reais. Esse enfoque melhora a capacidade das equipes de focar na remediação dos riscos mais impactantes para o negócio.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, a sua compreensão sobre a segurança de aplicações foi expandida do código para o comportamento em tempo de execução. Você aprendeu a diferença estratégica entre SAST (*Shift Left*) e DAST (*Shift Right*), reconhecendo que o DAST é essencial para capturar falhas de configuração e *runtime* que o SAST não consegue.

A análise se aprofundou nos mecanismos operacionais do OWASP ZAP, distinguindo o Passive Scan (rápido e não-intrusivo, ideal para *baseline*) do Active Scan (intrusivo, para *pentest* em ambientes isolados). Além disso, foram apresentadas técnicas avançadas de varredura autenticada, que usam *frameworks* de teste funcional para guiar o ZAP por áreas restritas.

Finalmente, o conteúdo destacou a integração prática do DAST via GitHub Actions contra ambientes AWS e a importância crítica do ciclo *Fix & Verify* para garantir a melhoria contínua da postura de segurança.

REFERÊNCIAS

CYCODE. SAST vs. DAST: **A Comprehensive Guide**. 2025. Disponível em: <https://cycode.com/blog/sast-vs-dast-guide/>. Acesso em: 28 nov. 2025.

JIT. **How to Automate OWASP ZAP**. 2024. Disponível em: <https://www.jit.io/resources/owasp-zap/how-to-automate-owasp-zap>. Acesso em: 28 nov. 2025.

OWASP. **OWASP ZAP Documentation**. 2025. Disponível em: <https://www.zaproxy.org/docs/>. Acesso em: 28 nov. 2025.

VEHENT, JULIEN. **Securing DevOps: Security in the Cloud**. Shelter Island: Manning Publications, 2018.

PALAVRAS-CHAVE

DAST. OWASP ZAP. Runtime Security. DevSecOps. GitHub Actions. Active Scan.
Passive Scan. Varredura Autenticada. AWS. CI/CD.

EMANIP



POS TECH